

WasteReporting

Documento Técnico: Arquitetura de Software e Pipeline CI/CD

Integrantes:

Rodrigo Belarmino de Oliveira
Murillo Cardoso Teixeira
Alexandre Coelho dos Santos Brito
Aldair Schmitberger Junior
Ygor Silva Dias de Carvalho

1. Descrição do Pipeline (CI/CD)

O processo de Integração e Entrega Contínua (CI/CD) foi implementado para automatizar o ciclo de vida de desenvolvimento, garantindo rapidez e confiabilidade nas entregas da aplicação.

- **Ferramenta Utilizada:** GitHub Actions.
- **Lógica de Modularização:** O pipeline foi estruturado de forma modular em dois arquivos distintos (`ci.yml` e `cd.yml`). Esta abordagem garante a manutenibilidade e permite a reutilização de etapas específicas do fluxo de trabalho.

Etapas do Fluxo

1. **Build:** Realiza a compilação da aplicação desenvolvida em **.NET 9** utilizando um ambiente Linux, garantindo a integridade do código fonte.
2. **Tests:** Execução de testes unitários com os frameworks **xUnit** e **Mock**. Esta etapa é fundamental para validar as regras de negócio antes de qualquer publicação.
3. **Publish:** Geração dos artefatos (binários) necessários para o deploy.
4. **Deploy:** Publicação automatizada no **Azure Web Apps**. O fluxo de deploy segue uma estratégia de branches:
 - **Staging:** Deploy automático via branch `develop`.
 - **Produção:** Deploy automático via branch `main`.

2. Docker: Arquitetura e Implementação

A estratégia de infraestrutura foi baseada em tecnologias de suporte a containers para garantir a consistência entre ambientes e otimização de recursos.

2.1. Arquitetura: Multi-stage Build

A arquitetura do Dockerfile utiliza a estratégia de **Multi-stage Build**, dividida em quatro estágios distintos para otimizar o resultado final:

- **Base:** Define o ambiente leve de execução utilizando o Runtime ASP.NET 9.0.
- **Build:** Utiliza o SDK completo (9.0) para baixar dependências (`dotnet restore`) e compilar o projeto.
- **Publish:** Prepara os arquivos finais otimizados através do comando `dotnet publish`.
- **Final:** Copia apenas os binários gerados para a imagem de runtime, descartando o SDK.

Vantagem: Esta abordagem garante uma imagem final extremamente reduzida, o que diminui a superfície de ataque e acelera o tempo de deploy, uma vez que o SDK pesado não é incluído no artefato de produção.

2.2. Comandos Utilizados

Os principais comandos utilizados no ciclo de desenvolvimento e orquestração são:

- `docker compose up --build`: Constrói as imagens e inicia todos os serviços (API e Orquestração) definidos.
- `docker compose down`: Para e remove os containers e redes criados.
- `docker build -t waste-reporting-api .`: Comando utilizado internamente pelo pipeline para gerar a imagem da aplicação.

2.3. Detalhes da Imagem e Infraestrutura

- **Base da Imagem:** Utiliza `mcr.microsoft.com/dotnet/aspnet:9.0` (baseada em Linux para alta performance).
- **Exposição de Portas:** A imagem escuta internamente a porta **8080**, seguindo a convenção moderna do .NET 9.
- **Redes:** Implementada uma rede do tipo *bridge* chamada `eth-network` para isolar o container da aplicação.
- **Volumes:** Mapeado um volume local `./logs` para a persistência dos logs gerados pela aplicação no caminho `/app/logs`.
- **Configuração:** O ambiente é totalmente configurável via variáveis de ambiente injetadas por arquivos `.env`.

3. Prints

This screenshot shows the GitHub Actions interface for the 'rb-belarmirno / WasteReporting' repository. The 'Actions' tab is selected, displaying a list of workflow runs. The left sidebar contains navigation links for 'All workflows', 'Continuous Deployment', 'Continuous Integration', 'Management', 'Caches', 'Deployments', 'Attestations', 'Runners', 'Usage metrics', and 'Performance metrics'. The main area shows 17 workflow runs, including 'docs: trigger final production deployment for documentation evidence #13', 'docs: clean up formatting and improve readability of README...', 'Merge pull request #4 from rb-belarmirno/develop', and 'docs: reorganize and expand technology stack documentation...'. Each run entry includes its name, commit hash, branch, status, and duration.

This screenshot shows the details of a specific workflow run: 'docs: trigger final production deployment for documentation evidence #13'. The workflow is currently 'In progress'. The left sidebar shows the 'Summary' tab, with links for 'All jobs', 'call-ci', 'build-and-test', 'Run details', 'Usage', and 'Workflow file'. The main area displays the workflow graph, showing a sequence of jobs: 'call-ci / build-and-test' (21s) followed by 'deploy-production' and 'deploy-staging'. The 'cd.yml' file is shown as the source of the workflow.

Chrome File Edit View History Bookmarks Profiles Tab Window Help

github.com/rb-belardino/WasteReporting/actions/runs/2468977272/job/72208379850

Summary

All jobs

- call-ci
- build-and-test

Run details

- Usage
- Workflow file

call-ci / build-and-test

Started 27s ago

Search logs

Build 11s

```
6 WasteReporting.API ->
/home/runner/work/WasteReporting/WasteReporting/src/WasteReporting.API/bin/Release/net9.0/WasteReporting.API.dll
7 WasteReporting.Tests ->
/home/runner/work/WasteReporting/WasteReporting/src/WasteReporting.Tests/bin/Release/net9.0/WasteReporting.Tests.dll
8 Build succeeded.
9 0 Warning(s)
10 0 Error(s)
11 Time Elapsed 00:00:09.15
```

Test

```
1 ▶Run dotnet test WasteReporting.sln --no-build --configuration Release --verbosity normal
6
7 Build started 04/20/2026 20:52:18.
8 1>Project "/home/runner/work/WasteReporting/WasteReporting/WasteReporting.sln" on node 1 (VSTest target(s)).
9 1>ValidateSolutionConfiguration:
   Building solution configuration "Release|Any CPU".
10 Test run for /home/runner/work/WasteReporting/WasteReporting/src/WasteReporting.Tests/bin/Release/net9.0/WasteReporting.Tests.dll
   (.NETCoreApp,Version=v9.0)
11
   Publish API
   Upload Artifact
   Post Setup .NET
   Post Run actions/checkout@v4
```

Chrome File Edit View History Bookmarks Profiles Tab Window Help

github.com/rb-belardino/WasteReporting/actions/runs/2468977272

Continuous Deployment

docs: trigger final production deployment for documentation evidence #13

Cancel workflow

Summary

All jobs

- call-ci
- deploy-staging
- deploy-production

Run details

- Usage
- Workflow file

Triggered via push 1 minute ago

rb-belardino pushed -> caabc18 main

Status: In progress

Total duration: =

Artifacts: 1

cd.yml

on: push

call-ci / build-and-test 33s

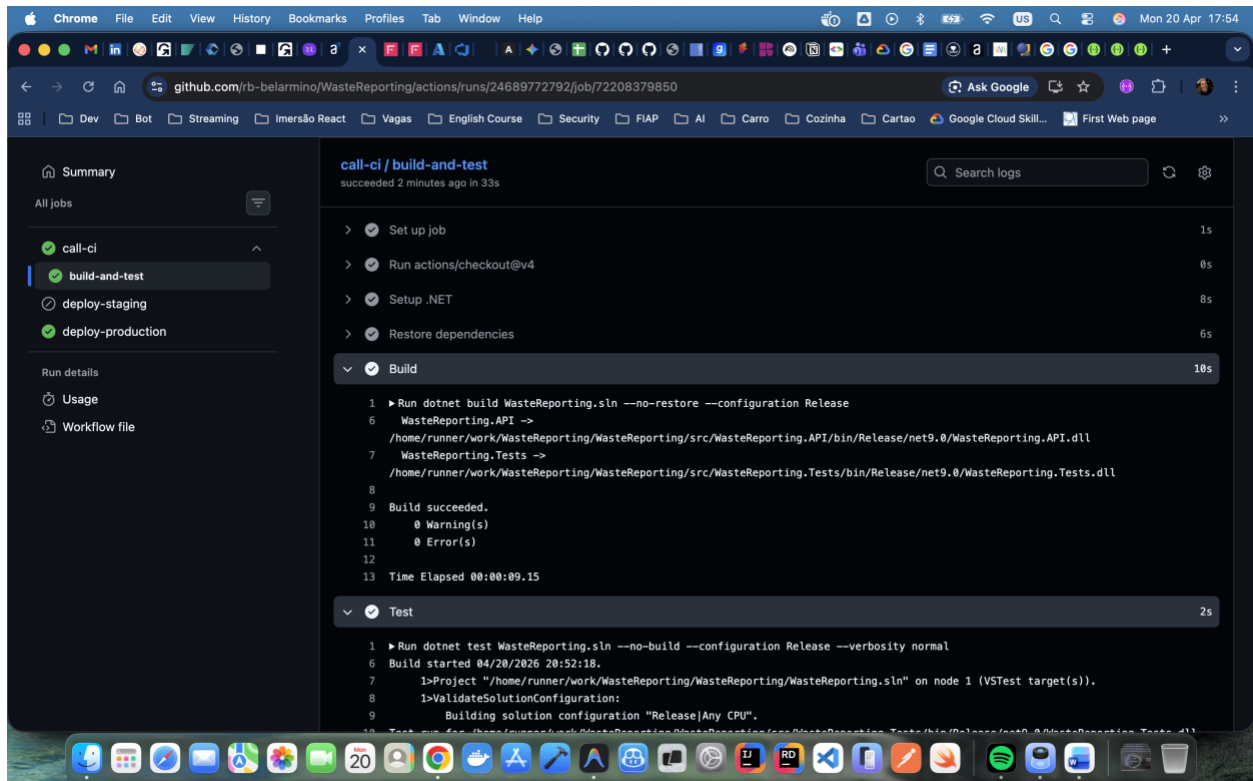
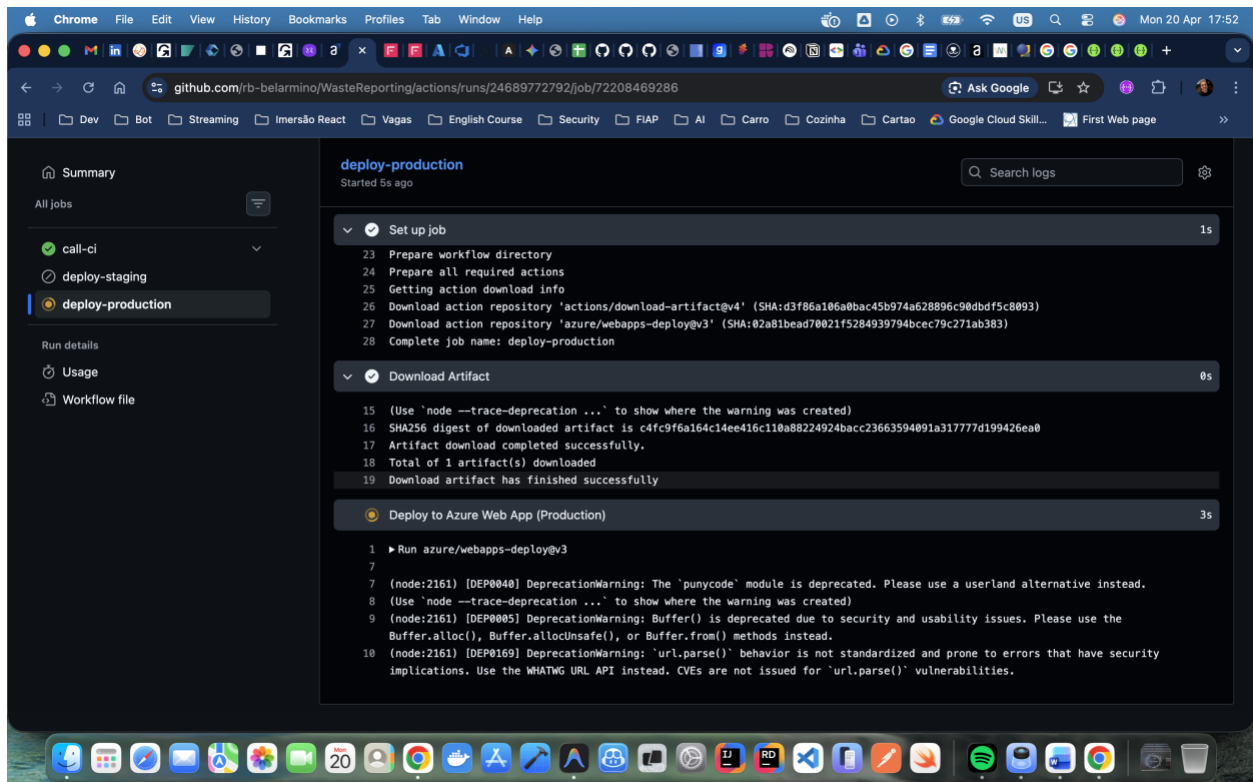
deploy-production 26s

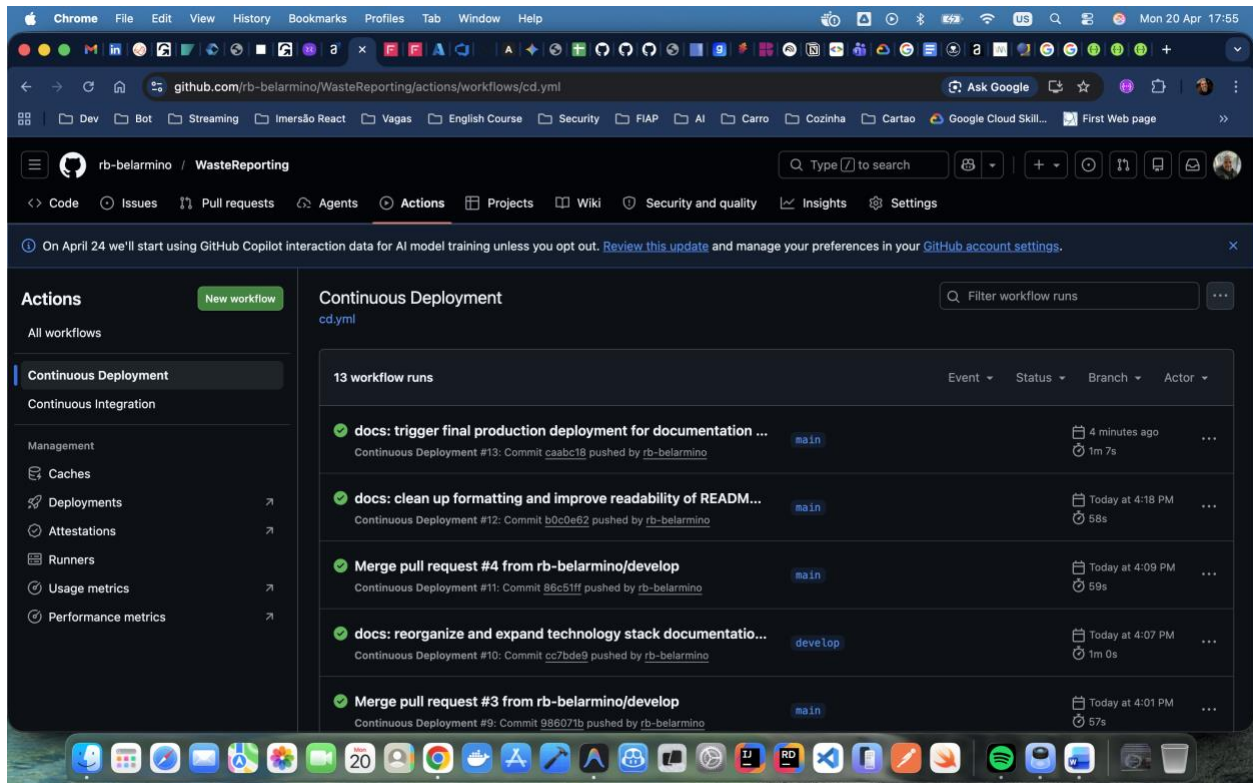
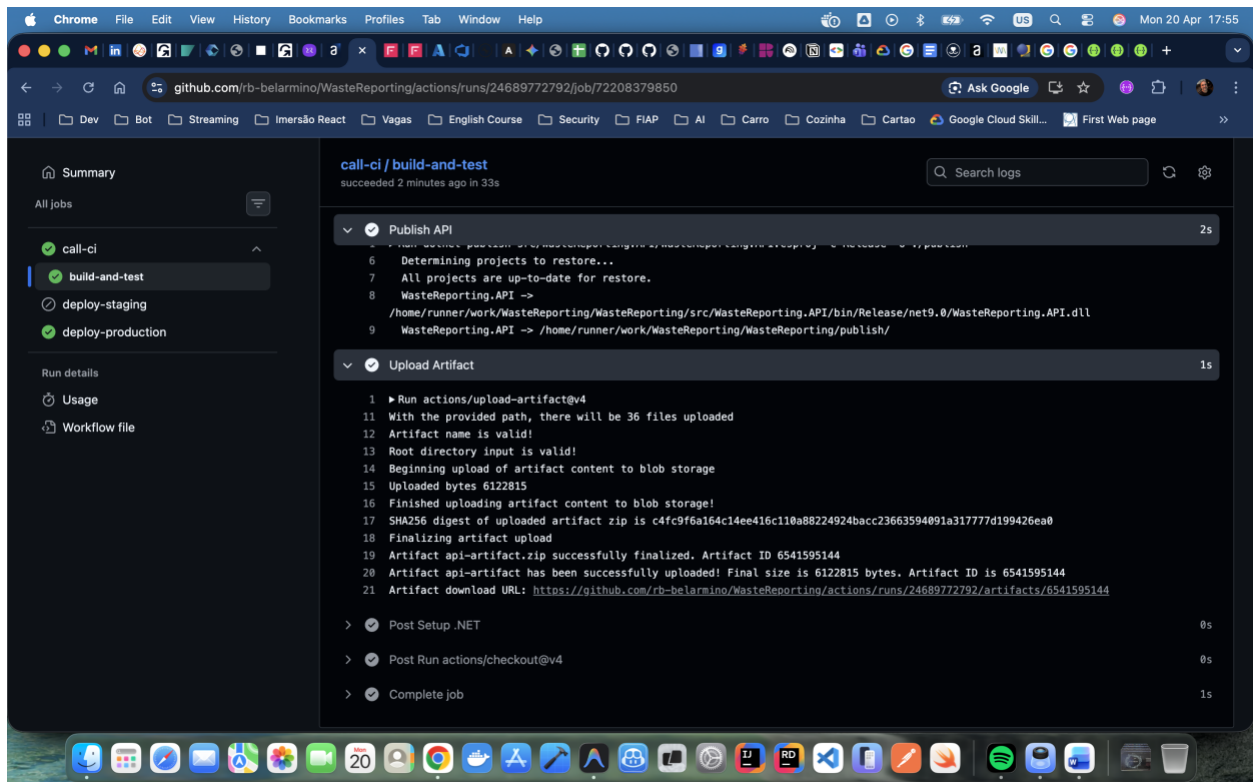
Deploying to production

deploy-staging 0s

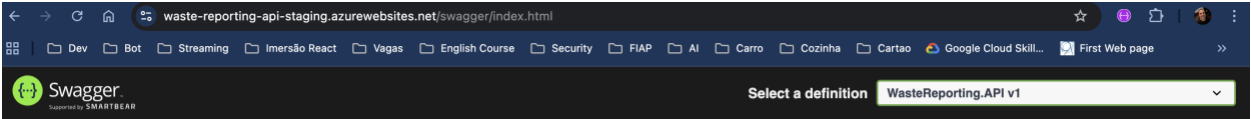
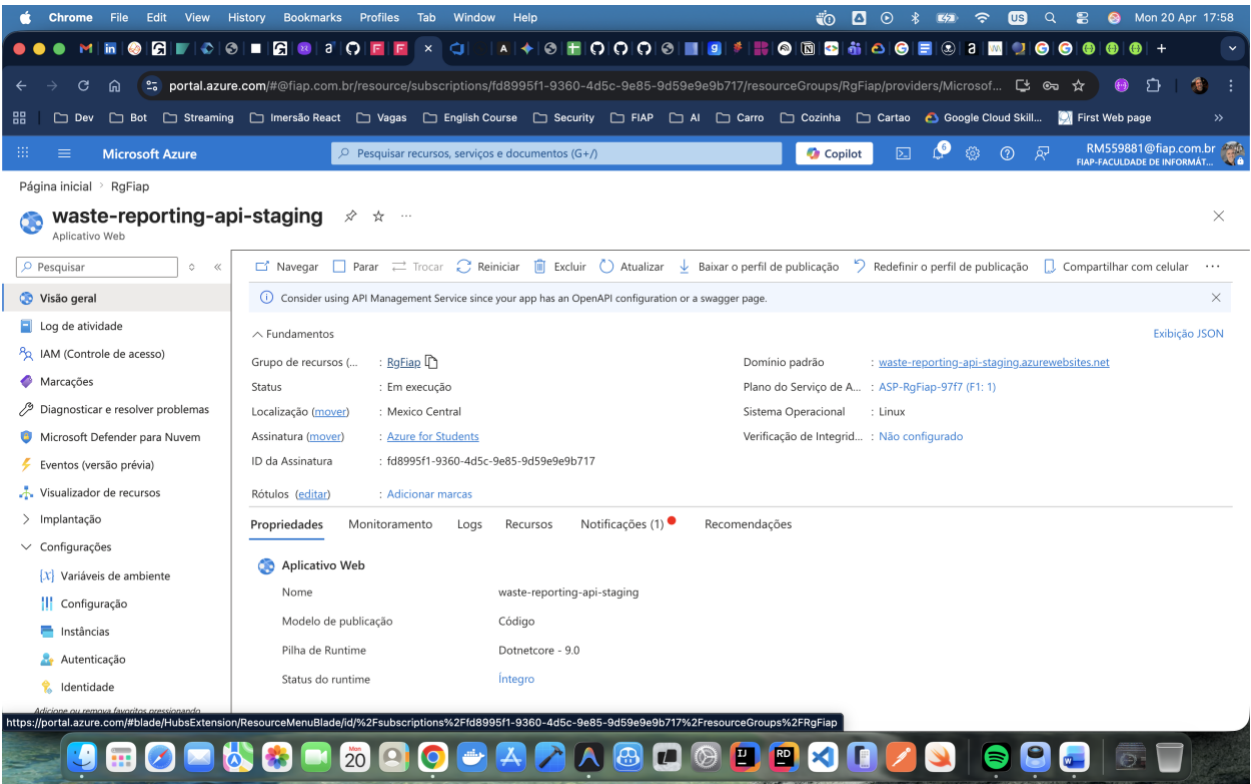
Annotations

1 warning





Ambiente Staging



Waste Reporting API v1 OAS3

<https://waste-reporting-api-staging.azurewebsites.net/swagger/v1/swagger.json>

Authorize

Auth

POST /auth/register Registers a new user.

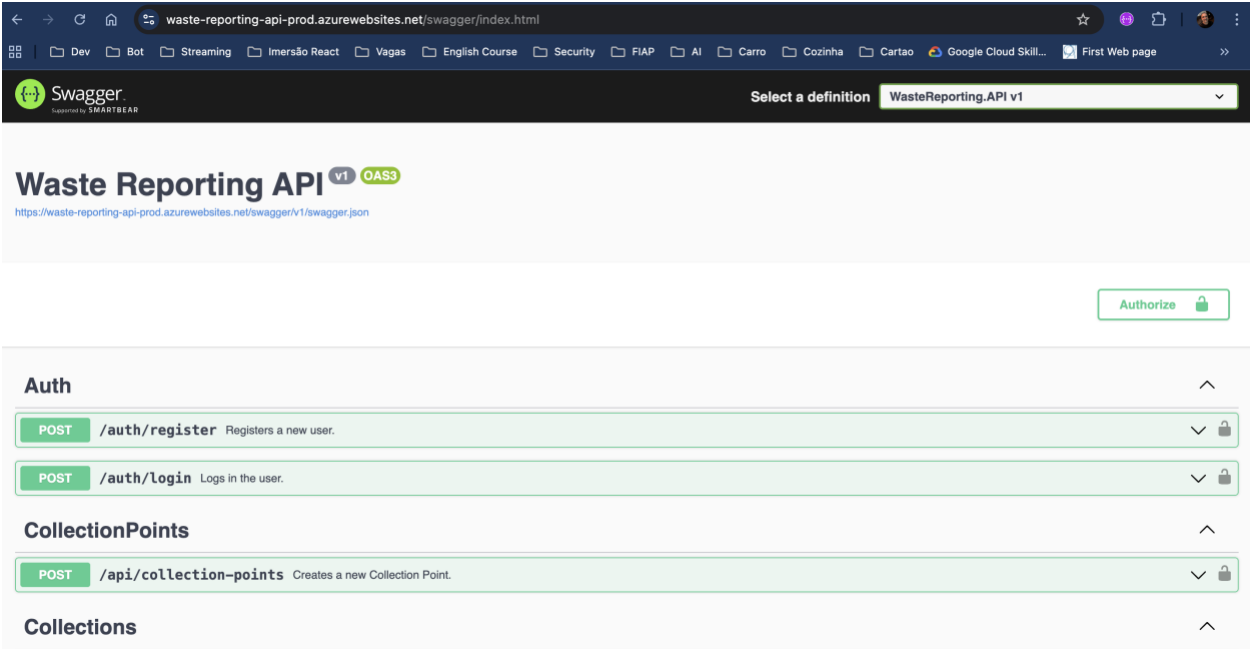
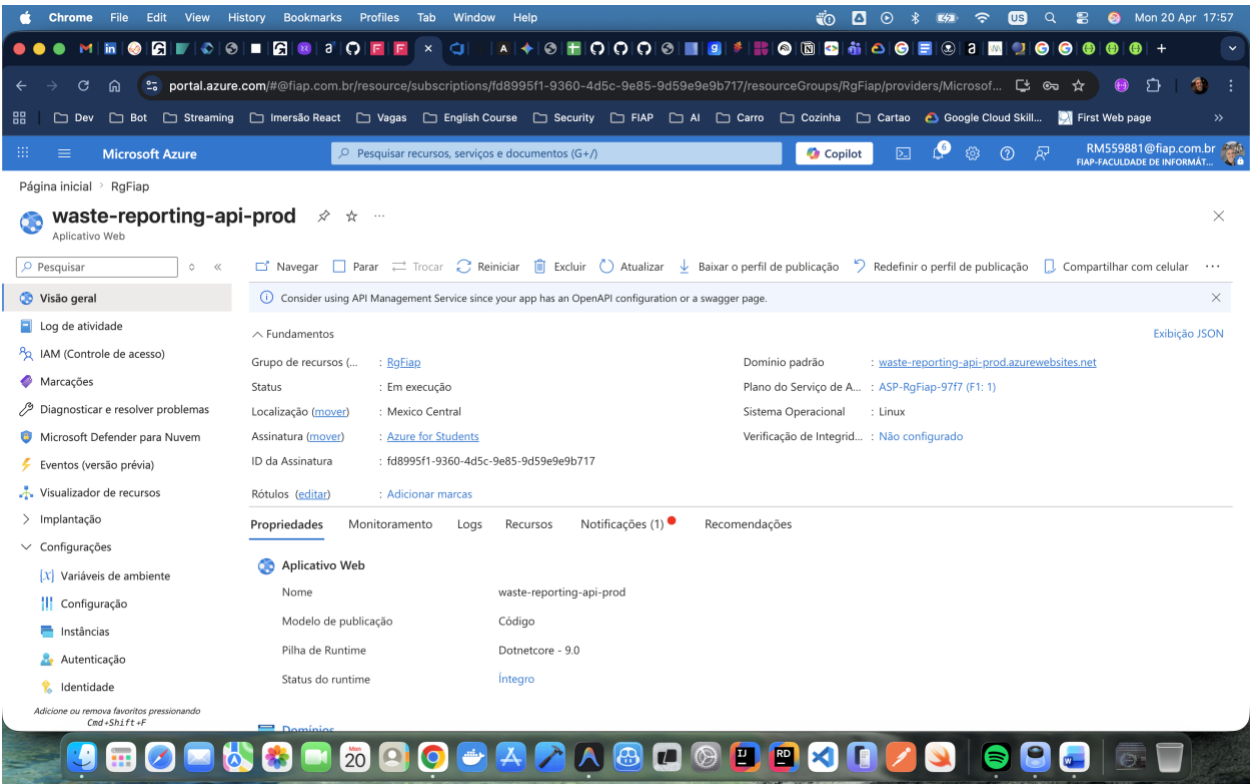
POST /auth/login Logs in the user.

CollectionPoints

POST /api/collection-points Creates a new Collection Point.

Collections

Ambiente de Produção



4. Desafios e Soluções

Desafio: Redundância e Complexidade na Pipeline de CI/CD:

Ao gerenciar dois ambientes distintos (Staging e Produção), o desafio inicial era evitar a duplicação de scripts de integração. Ter que configurar as etapas de recompilação (Build) e execução de testes em múltiplos arquivos YAML tornava o processo sujeito a erros e difícil de manter, além de aumentar o tempo de manutenção da pipeline.

Solução: Implementação de Workflows Reutilizáveis (GitHub Actions)

A solução adotada foi o desacoplamento da pipeline em módulos reutilizáveis através do recurso *workflow call*.

1. Criamos um workflow base de **Integração Contínua (CI)** focado puramente em qualidade (Build + Testes).
2. Configuramos o workflow de **Entrega Contínua (CD)** para "chamar" esse módulo de CI antes de prosseguir com qualquer deploy.
3. **Resultado:** Garantimos que o código que chega em **Staging** é validado pela mesma régua de testes que o código de **Produção**, reduzindo a duplicidade de código em 50% e centralizando todas as regras de **build** em um único ponto da arquitetura.

4. Checklist de Entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☒
Dockerfile funcional	☒
docker-compose.yml ou arquivos Kubernetes	☒
Pipeline com etapas de build, teste e deploy	☒
README.md com instruções e prints	☒
Documentação técnica com evidências (PDF ou PPT)	☒
Deploy realizado nos ambientes staging e produção	☒